复旦大学计算机科学技术学院

学年第　学期期末试卷

课程名称：＿＿＿＿＿＿＿＿＿＿　　课程代码：＿＿＿＿＿＿＿＿

卷　　别：■A 卷　　□B 卷　　□C 卷

姓　　名：＿＿＿＿＿＿＿＿＿＿　　学　　号：＿＿＿＿＿＿＿＿

提示：请同学们秉持诚实守信宗旨，谨守考试纪律，摒弃考试作弊。学生如有违反学校考试纪律的行为，学校将按《复旦大学学生纪律处分条例》规定予以严肃处理。

（装订线内不要答题）

题号	一			二		三	四			总分
	1	2	3	1	2	1	1	2	3	
得分										

一、阅读程序写输出（30%）

第 1 题（10%）

```
a = 10
b = 20
if a > b:
   print(a)
elif a <= b // 2:
   print(b // 2)
if a > b / 2:
   print(a)
else:
   print(b / 2)
```

第 2 题（10%）

```
for i in range(4):
    for j in range(i, -1, -1):
        if (i+j)%2==0:
            print(i, j)
```

第 3 题（10%）

```
aList = list("python 期末考试"[:-4])    #注：字符串中无空格
print(",".join(aList))
aList[1::2] = list("测试题")
print(",".join(aList))
aList[::] = ["all"]
print(aList)
```

二、程序填空（20%）

第 1 题（10%）

```
#  -*- coding: utf-8 -*-
'''-------------------------------------------------------
【程序填空】
-------------------------------------------------------
根据程序中注释，完成下列程序，使得该程序运行结果如下图所示。
-------------------------------------------------------
注意:
(1) 每行限填一行代码。
(2) 除要求填空的位置之外，请勿改动程序中的其他内容。
-------------------------------------------------------'''

Total = 8 # 当前已筛选学生的总数
PassStudents = { "Alice":85,"Kate":65} #当前及格的同学，分数>=60

def ___________1_____________:
  """
    从列表 students 中筛选分数超过 grade 分数的学生,添加到字典 PassStudents 中，
    同时增加已筛选学生的总数 Total。【注：学生的名字不会重复】
  """
  ____________2____________
  for student in students:
      Total += 1
      if student[1] >= grade:
          PassStudents[student[0]] = student[1]

def main( ):
    students = [("Mary", 40), ("Bob", 90)]#将要被筛选学生的列表
    filter_students(students)
    print("当前共筛选%d 位同学，及格的同学有:%s"
          % (Total,",".join(PassStudents.keys())))

if __name__ == "__main__":
```

```
    main()
```

程序的运行结果如下图所示：

```
当前共筛选10位同学，及格的同学有:Alice,Kate,Bob
```

第 2 题（10%）

```
#  -*- coding: utf-8 -*-
'''-----------------------------------------------------
```

下面的程序根据步行记录列表 mylist 中的信息，计算出的步行总数最多的月份以及相应的总步数。列表 mylist 的每个元素是一个元组，元组的第一项是“月-日”形式的日期、第二项是相应的步数。

使得该程序运行结果如下图所示。

```
-----------------------------------------------------
```

注意：

(1) 每行限填一行代码。

(2) 除要求填空的位置之外，请勿改动程序中的其他内容。

```
-----------------------------------------------------'''
```

```
mylist= [('9-30','12305'), ('10-4','11056'), ('9-15', '9815'), ('10-28', '11521'), ('8-09', '6831'), ('08-16', '8889'), ('09-17', '13866')]

steps_dict = {}
for record in mylist:
    date = record[0]
    step = record[1]
    date = date.rstrip('_____1_____')    #完成此处的程序填空
    month = int(date[0:len(date)-1])
    curstep = steps_dict.get(month, 0)
    steps_dict[month] = curstep + int(step)

max_steps = 0
max_month = None
for month, total_steps in steps_dict.items():
    if total_steps > max_steps:
        max_steps = total_steps
        max_month = month

result = "_____2_____"%(max_month, max_steps)    #完成此处的程序填空
print(result)
```

要求完成程序填空，使得上述程序的输出如下结果：

```
步数最多的月份是9月，步数是35986。
```

三、程序改错（10%）

21 点游戏。有 52 张纸牌，牌面上的数字就是点数，特殊地，J、Q、K 都算 10 点，A 算 11 点。当抓了 n 张牌后，如果点数之和 X 小于 21 点，则需判断是否还要抓一张。策略是：假设 Y=21-X，如果剩余牌中大于 Y 的纸牌数量>=小于 Y 的纸牌数量，则停止抓牌，否则继续抓一张。

程序运行的两个示例如下：

```
请输入抓牌的张数:6
2
3
2
3
2
3
STOP
```

[解释] 已抓 6 张牌的总点数是 15，与 21 的差是 6。剩余牌中比 6 大的牌数>小于 6 的牌数，所以不再继续抓牌,输出 STOP。

```
请输入抓牌的张数:1
10
DRAW
```

以下程序有两处错误，请在答题位置填写错误行的编号(每行最右边注释里的数字。如果代码行后面没有注释，表示该行不会出错)和改正后的整行代码。改正后的代码只能是一行，不允许跨越多行。

```
points_num = [4] * 10    # 点数 2~11 对应的牌数
points_num[-2] = 16    # 特殊地，点数为 10 的牌有 16 张
X = 0                                                                    # 1
n = int(input("请输入抓牌的张数:"))                                        # 2
for i in range(1, n):                                                    # 3
    points = int(input())                                                # 4
    X += points                                                          # 5
    points_num[points-2] -= 1    # 与点数对应的牌数减一                   # 6
Y = 21 - X                                                               # 7
greater_num = [num for points, num in enumerate(points_num) if points > Y]   # 8
if sum(greater_num) >= sum(points_num) - sum(greater_num):               # 9
    print('STOP')                                                        # 10
else:                                                                    # 11
    print('DRAW')                                                        # 12
```

四、编程（40%）

第 1 题（10%）

编写程序实现如下功能。

字典 students 表示各门课的选修学生，下面给出了一个示例：

```
students = {
    "physics": ["Zora", "Lucas"],
    "chemistry": ["Martin", "Zora", "Sam"],
    "geography": ["Sam", "Wendy"],
    "biology" : ["Martin", "Zora"],
    "history": ["Sam", "Martin", "Wendy", "Lucas"],
    "politics": ["Lucas", "Wendy"]
}
```

程序根据 students 中的选课信息，输出每位学生的所选课程的情况。对于上述的 students，其输出如下所示，注意学生姓名及其所选课程名均按升序排列。

```
Lucas:history,physics,politics
Martin:biology,chemistry,history
Sam:chemistry,geography,history
Wendy:geography,history,politics
Zora:biology,chemistry,physics
```

程序已经有如下代码：

```
students = {
    "physics": ["Zora", "Lucas"],
    "chemistry": ["Martin", "Zora", "Sam"],
    "geography": ["Sam", "Wendy"],
    "biology" : ["Martin", "Zora"],
    "history": ["Sam", "Martin", "Wendy", "Lucas"],
    "politics": ["Lucas", "Wendy"]
}
```

请在答题纸上完成其余部分。

第 2 题（10%）

```
#   -*- coding: utf-8 -*-
'''----------------------------------------------------------------
【程序设计】
----------------------------------------------------------------
```

text 是某个系统的部分运行记录，包含两种格式：

1）事件开始记录：以"小时:分钟:秒,毫秒"为时间，后接 7 位数字为事件唯一编号，以"start"结尾;

2）事件结束记录：时间为"小时:分钟:秒,毫秒"，后接 7 位数字作为事件唯一编号，然后是"finish 0"或"finish 1"。前者代表普通事件结束，后者代表异常事件结束。

该系统一次只处理一个事件，text 中每两行对应着一个事件，第一行是事件的开始记录，下一行是事件结束记录。事件持续时间是同一事件的事件结束和事件开始时间的差值。下面的

Python 代码计算所有异常事件持续时间的总和，并求出平均值。函数 parse_time 把上述格式的时间转换为以秒为单位的时间，并在 main 函数中适当地调用。输出结果保留 2 位小数，不能使用 re 库。程序运行后输出

异常事件 总时间: 4.66s 平均时间: 1.55s

注意：部分源程序给出如下。请勿改动已给出的代码，

仅在注释行 Program 和 End 之间填入所编写的若干语句。

---'''

```
text = '''15:12:21,681 2331947 start
15:12:22,687 2331947 finish 1
15:12:22,694 2331955 start
15:12:25,864 2331955 finish 0
15:12:26,383 2331975 start
15:12:28,206 2331975 finish 1
15:12:55,880 2332210 start
15:12:57,708 2332210 finish 0
15:13:02,654 2332264 start
15:13:04,482 2332264 finish 1'''

def parse_time(s):
    '''把时间串 HH:MM:SS,fff 转换成以秒为单位的时间，以浮点数表示''
    #**********Program**********




    #**********  end  **********

def main():
    lines = [ line.split() for line in text.split('\n')]
    count, total_time = 0, 0
    #**********Program**********




    #**********  end  **********
    print("异常事件 总时间:",round(total_time,2), "平均时间:", round(total_time/count,2))


if __name__ == "__main__":
    main()
```

第 3 题（20%）

```
#  -*- coding: utf-8 -*-
'''------------------------------------------------------
【程序设计】
------------------------------------------------------
```

分析一个文本文件，找到其中那些满足下述格式的行，该行包含如下部分：

- 0 到多个空格开始
- [月份.日期]。月份和日期为 1 到 2 位数字
- 若干字符
- <一系列的整数>。通过空格将这些整数隔开，注意这些整数前后都有可能会有空格，而且空格的个数可能有多个
- 0 到多个空格结束

获取其中的月份和日期，以及那些整数。如果某个整数连续出现，则仅仅保留 1 个。

文本文件 file.txt（编码方式为 UTF-8)的部分内容如下所示：

```
# 下面为某个程序运行时产生的日志的一部分内容：
2023-11-12 18:40:01 From [192.168.1.1], Info:
    [8.12] Tony {v1} Tom 120 <01 1 42 55 55 55 1       78 >
[8.13] Tony Bob 120          <    12 4 42 55 55 55 124 124 55>
[8.14] Tony Bob Mary 123
2023-11-21 08:40:01 From [192.168.1.5], Information:
[09.1] <123>
[9.02] James Bob 120        <12345      5       777    777    777>
```

请实现两个函数：

- uniq(numbers)。numbers 为包含若干整数的列表（假设至少包含一个整数）。返回一个新的列表，该列表的元素来自于 numbers，只是去除了 numbers 中那些连续出现的重复整数，即如果某个整数连续出现，则仅仅保留 1 个。
- parse(name)。name 为文件名。对该文件的内容进行分析处理，从那些满足指定格式的行中获取相关信息，调用上述 uniq 函数去除重复整数，最终返回一个列表，其中的每个元素对应着指定格式的行中的数据。如果没有找到指定格式的行，则抛出异常 ValueError，给出原因为 not found。对于上述文件，调用函数 parse('file.txt') 分析处理后，最终返回的对象为：
  [(8, 12, [1, 42, 55, 1, 78]), (8, 13, [12, 4, 42, 55, 124, 55]), (9, 1, [123]), (9, 2, [12345, 5, 777])]

```
------------------------------------------------------------
注意：部分源程序给出如下。请勿改动已给出的代码，
      仅在注释行 Program 和 End 之间填入所编写的若干语句。
------------------------------------------------------------'''
import re
#**********Program**********
```

```
#**********  End  **********
if __name__ == '__main__':
    records = parse(text)
    print(records)
```